BÖLÜM 3: VERI MIMARISI — DURUMUN YAŞADIĞI, ÖLDÜĞÜ VE YALAN SÖYLEDIĞI YER ⏱ 9 dk • DERS 11

Parça 11: ACID vs. BASE Karşılaşması: Gerçek Dünyada Tutarlılık Modelleri

CAP Teoremini yönetmek: Mükemmelliği mi yoksa ayakta kalmayı mı seçiyorsunuz?

> 🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**
>
> Bir dijital cüzdan girişimi, yüksek işlem hacmini kaldırabilmek için BASE semantiğine sahip (Eventually Consistent) bir NoSQL veritabanı seçti. Bir Cuma akşamı bir üniversite öğrencisi, ATM'den 200 TL çekerken aynı saniyede mobil uygulamadan arkadaşına 200 TL transfer ettiğinde her iki işlemin de başarılı olduğunu keşfetti; çünkü dağıtık düğümler henüz bakiyeyi eşitlememişti! Hile TikTok'ta yayıldı. 48 saat içinde 10.000 kullanıcı sistemden 3.2 milyon TL çekti. Şirket API'yi kapatmak ve acil sermaye artırımına gitmek zorunda kaldı.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

CAP teoremi (Tutarlılık, Erişilebilirlik, Bölünme Toleransı) akademik bir felsefe değil; dağıtık sistemlerin yerçekimi kanunudur. Ağda bir kopma veya gecikme olduğunda Sistem Tutarlılığı (yanlış veri vermemek için isteği reddetmek) ile Sistem Erişilebilirliği (veri bayat bile olsa isteğe yanıt vermek) arasında bir seçim yapmak ZORUNDASINIZ. Para işlemleri mutlak ACID ister; sosyal medyadaki beğeni sayıları ise BASE ile harika yaşar.

Temel Mimari İlkeler ve Zihinsel Modeller

ACID ve BASE Felsefesi

- **ACID (Geleneksel RDBMS):**
- **Atomicity:** Ya hep ya hiç.
- **Consistency:** Veritabanı kuralları daima korunur.
- **Isolation:** Eşzamanlı işlemler birbirini bozmaz.
- **Durability:** Onaylanan işlem elektrik kesilse de kaybolmaz.
- **BASE (Dağıtık NoSQL):**
- **Basically Available:** Sistem her zaman yanıt vermeye çalışır.
- **Soft State:** Veri arka planda senkronize olurken durum değişebilir.
- **Eventual Consistency:** Yeni güncelleme gelmezse tüm kopyalar eninde sonunda eşitlenir.

PACELC Teoremi (CAP'in Ötesi)

Bölünme (**P*artition) anında** A*vailability mi yoksa **C*onsistency mi seçiyorsun? Normal zamanda (**E*lse) ise **L*atency mi yoksa** C*onsistency mi seçiyorsun? Kıtalararası sistemlerde ışık hızını aşamazsınız; güçlü tutarlılık mutlaka gecikme vergisi ödetir.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Milisaniyelik Masalı:** 'Eventual' kelimesini '2 milisaniyede eşitlenir' sanıp ağ koptuğunda kopyaların 10 dakika geriden gelmesine hazırlıksız yakalanmak.

- **Kıtalararası 2PC Fantezisi:** Kıtalar arasında 2-Phase Commit ile dağıtık ACID sağlamaya çalışıp işlem sürelerini 800ms'ye fırlatmak.
- **İzolasyon Seviyesi Körlüğü:** Birçok veritabanında varsayılan seviyenin `Read Committed` olduğunu ve fantom okumalara izin verdiğini bilmeden 'nasılsa ACID' rahatlığına kapılmak.

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Verilerini ikiye ayır: 'Asla yanılmaması gerekenler' (Finans, Stok) vs. 'Geçici olarak bayat kalabilecekler' (İzlenme sayısı, Beğeniler).
> - **2. Adım:** Finansal çekirdeği güçlü ACID ilişkisel veritabanında tut.
> - **3. Adım:** Dağıtık servisler arasında ACID kilitleri yerine telafi edici işlemler (Saga) tasarla.

Mimari Çıkarımlar ve Hile Tablosu

- Fiziği yenemezsiniz: Dağıtık düğümlerde yüksek erişilebilirlik anlık tutarlılıktan ödün vermeyi gerektirir.
- Bakiye ve bilet mutlak tutarlılık ister; sosyal akış ve öneriler eventual consistency ile yaşar.
- Eventual consistency seçtiyseniz, çakışma anında iş biriminin ne yapacağını önceden planlamalısınız.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Ürününüzde güçlü tutarlılık kullanan bir özellik seçin. Bu veri 2 saniye gecikmeli gelseydi iş birimi gerçekten zarar görür müydü? Ne kazanırdınız?